

Análisis de la herramienta ESBMC

Tomás Achával Berzero, Mauricio Jeremías Llugdar y Gregorio Vilardo

Facultad de Matemática, Astronomía, Física y Computación
Universidad Nacional de Córdoba
`{tomasachaval, mauriciojllugdar, gregovilardo}@mi.unc.edu.ar`

Resumen ESBMC (Efficient SMT-based Context-Bounded Model Checker) es una herramienta de código abierto diseñada para la verificación de programas en múltiples lenguajes de programación. En este informe se analiza su historia y su estado actual, entrando en detalles técnicos y recopilando casos de estudio exitosos en la industria. ESBMC automatiza la detección de fallos críticos de memoria y otros problemas lógicos transformando el código fuente en fórmulas que son resueltas por solucionadores SMT.

1. Introducción

Este informe busca realizar un análisis general del estado actual de la herramienta de verificación de software ESBMC. Se cubrirán temas desde su contexto de creación hasta casos de estudio actuales, pasando por una explicación de su forma de utilización y algunos detalles técnicos.

Organización del informe. El resto de este trabajo se estructura de la siguiente manera. Las Secciones 2 y 3 introducen el contexto de creación y el objetivo de la herramienta. La descripción desde la perspectiva del usuario junto con los aspectos técnicos fundamentales se detallan en las Secciones 4 y 5. En las Secciones 6 y 7 se presentan casos de estudio y se realiza una comparación con otras herramientas. El análisis detallado de un caso de estudio particular se encuentra en la Sección 8. Finalmente, en la Sección 9 se expone nuestra conclusión.

2. Contexto de creación de la herramienta

La primera versión de ESBMC fue desarrollada en 2008 por Lucas Cordeiro (University of Southampton), Bernd Fischer (University of Southampton) y Joao Marques-Silva (University College Dublin) [3,4]. La herramienta fue construida sobre otra llamada CBMC, aunque con el tiempo casi todos los componentes han sido re-implementados [5]. Su motivación inicial fue pasar de utilizar solucionadores SAT a aprovechar las ventajas de los solucionadores SMT¹.

¹ Los solucionadores SAT y SMT son herramientas que deciden si una fórmula lógica puede ser verdadera. La ventaja de los SMT es que admiten fórmulas más complejas.

Desde aquel entonces hasta ahora, han recibido el apoyo de múltiples universidades y empresas, y el desarrollo es mayormente llevado a cabo por la comunidad. Actualmente es una herramienta completa que se utiliza para verificar programas en C, C++, CUDA, CHERI, Kotlin, Python, Rust, y Solidity, incluyendo concurrencia.

ESBMC continúa mejorando en un contexto académico, donde se mejoran los algoritmos utilizados y se extiende el soporte a más lenguajes. Además, se ha puesto a prueba frente a otras herramientas de verificación de software en múltiples ediciones de la competencia SV-Comp [2] desde 2012.

Al ser una herramienta de código abierto bajo una licencia permisiva, también se utiliza en la industria. Por ejemplo, recientemente se verificaron smart contracts en Solidity, aplicaciones de Android y se hallaron errores en la Ethereum Consensus Specification. A medida que se añade soporte para más lenguajes y se extiende el ya existente, crecen los ejemplos exitosos de aplicación en la industria.

3. Objetivo de la herramienta

El objetivo principal de ESBMC es verificar automáticamente la corrección de programas de software y encontrar vulnerabilidades lógicas o de seguridad antes del despliegue. El hecho de soportar múltiples lenguajes y concurrencia significa que puede aplicarse a una amplia gama de problemas.

Uno de sus mayores focos es la seguridad de memoria, muy útil en lenguajes como C: punteros colgantes, desreferencia de punteros nulos, desbordamientos de búfer, fugas de memoria, etc. En los lenguajes que ya son *memory-safe*² como Python, se puede utilizar para la detección de desbordamientos aritméticos, divisiones por cero, para verificar aserciones definidas por el usuario, entre otros usos. Además, es capaz de razonar sobre las estructuras orientadas a objetos, muy utilizadas en estos lenguajes.

Esto hace que sea una herramienta muy útil en las etapas de implementación, testing y mantenimiento de un proceso de desarrollo de software. ESBMC verifica problemas comunes automáticamente, provee funciones que el usuario puede incluir en su código mientras lo desarrolla para detectar errores y también asiste en la creación de tests (incluso generándolos automáticamente) y por ello es útil en todas las etapas mencionadas.

4. Descripción de la herramienta del lado del usuario

La herramienta se utiliza desde la línea de comandos ejecutando el binario `esbmc`, pasando el archivo de código fuente y diversas *flags* para especificar el proceso de verificación que se desea llevar a cabo.

² Un lenguaje *memory-safe* garantiza que sus programas solo acceden a ubicaciones de memoria autorizadas.

Las *flags* permiten seleccionar el solucionador SMT a utilizar, entre los cuales se encuentran Bitwuzla, Boolector, Z3, MathSAT, CVC4 y Yices. Si se desea utilizar otro solucionador se puede indicar con `--smtlib --smt-formula-only --output` y ESBMC generará un archivo compatible con el estándar SMT-LIB v2, el cual es soportado por la mayoría de los solucionadores SMT. Mediante ellas también se pueden solicitar verificaciones de propiedades específicas (p.ej. `--memory-leak-check`) y se puede seleccionar la estrategia de verificación que se desea seguir (p.ej. `--k-induction`). Por último, provee *flags* y *construcciones* para la generación automática de tests en algunos de los lenguajes soportados (p.ej. `--branch-coverage --generate-ctest-testcase` en C).

ESBMC trabaja directamente sobre el código fuente del programa sin necesidad de anotaciones o modificaciones. Puede verificar automáticamente propiedades como accesos fuera de rango, división por cero, overflow, errores de memoria, deadlocks o race conditions. Si el usuario desea definir propiedades más específicas sobre su programa, ESBMC provee herramientas como construcciones (p.ej. `__ESBMC_assert()`, `nondet_int()`) y contratos de funciones (p.ej. `__ESBMC_requires(arg ≥ 0)`) que se pueden incluir en el código fuente. Con ellas se pueden definir propiedades que dependen de variables (p.ej. `__ESBMC_assert(discount ≤ price)`), pre y post condiciones de funciones, se puede informar al verificador sobre los valores que tomará una variable, etc.

Todo esto permite verificar propiedades tanto de safety como de liveness tales como deadlocks, aserciones y propiedades definidas por el usuario, propiedades de seguridad de memoria, desbordamientos aritméticos, y más. En caso de que se alcance un estado no deseado (es decir, se detecta una condición de carrera, se detecta una posible violación de una aserción del usuario, etc.), ESBMC generará un contraejemplo que indica exactamente dónde y cómo puede ocurrir el problema.

5. Aspectos técnicos de la herramienta

Para lograr los objetivos discutidos anteriormente, ESBMC representa el espacio de estados de manera simbólica, donde el producto final es la fórmula SMT, que luego será analizada por un solucionador SMT seleccionado.

Para lograr la fórmula deseada, en primera instancia se utiliza Clang como frontend para delegar la creación del árbol de sintaxis abstracta (AST), característica introducida en la versión 7.3 [11]. Para los lenguajes no soportados por Clang, se utilizan frontends específicos que se pueden ver en la Figura 1. Luego, el generador de CFG (Control Flow Graph) transforma el AST directamente en un GOTO program, que es una representación equivalente pero simplificada: solo contiene asignaciones, saltos condicionales e incondicionales (GOTO), assumptions y assertions. Esta representación es la que utiliza ESBMC como su grafo de flujo de control concreto. Posteriormente, el motor de ejecución simbólica ejecuta este programa y genera la Asignación Única Estática (SSA) tras haber desenrollado los loops k veces [8,9].

A partir de la SSA, ESBMC genera la fórmula SMT final. Sin embargo, para evitar la explosión del espacio de estados en programas complejos o con bucles infinitos, la herramienta implementa técnicas de abstracción y refinamiento. Por un lado, utiliza Modelos Operacionales (OM), que son abstracciones matemáticas de las bibliotecas estándar (como POSIX o la STL de C++) que reemplazan el código real por pre y postcondiciones más simples de verificar. Por otro lado, para superar la limitación del desenrollado acotado, ESBMC incorpora el algoritmo de k -inducción. Esta técnica intenta probar propiedades de manera inductiva sobre los bucles. Si el paso inductivo falla debido a estados inalcanzables, la herramienta refina el análisis mediante el fortalecimiento de invariantes (invariant hardening) o incrementando el valor de k .

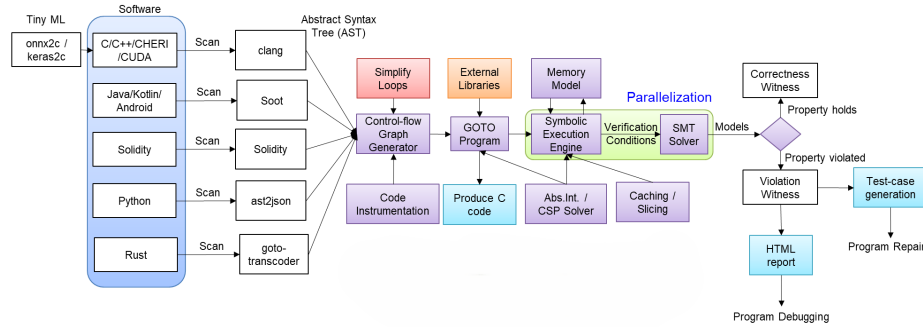


Figura 1. Arquitectura de ESBMC.

En cuanto a la consideración del espacio de estados, el comportamiento de ESBMC varía según su modo de configuración:

- **Modo BMC puro (Acotado):** Considera únicamente una fracción del espacio de estados (el prefijo de ejecución hasta la cota k). En este modo, la respuesta es exacta pero incompleta: no admite falsos positivos (cualquier error encontrado es un bug real y reproducible). Sin embargo, puede dar lugar a falsos negativos si el fallo ocurre a una profundidad mayor que k . Esto sucede porque BMC sub-aproxima el espacio de estados: al desenrollar los bucles solo k veces, cualquier traza que necesite más de k iteraciones para infringir la propiedad queda fuera del prefijo que se ha explorado. Es una limitación que forma parte de la verificación acotada, no un defecto de la herramienta.
- **Modo K-Inducción (No Acotado):** Si el proceso inductivo converge, ESBMC analiza la totalidad del espacio de estados, permitiendo certificar la corrección matemática del programa (cero falsos negativos). No obstante, si las abstracciones de los modelos operacionales o los invariantes generados son demasiado imprecisos, este modo puede introducir falsos positivos (alertas de fallos que no son realizables en el código real).

5.1. Fórmula BMC y algoritmo de K-Inducción

El núcleo de la funcionalidad de ESBMC es la técnica de Verificación de Modelos Acotados (BMC), en donde el programa se modela como un sistema de transición de estados que deriva del CFG, donde los vértices eran las asignaciones o sentencias condicionales, y las aristas eran los posibles cambios en el control de flujo del programa.

Sea $M = (S, S_0, T)$ el sistema de transiciones, donde S es un conjunto no vacío de estados (cada estado representa una configuración completa del programa en un instante dado), $S_0 \subseteq S$ es el conjunto de estados iniciales, y $T \subseteq S \times S$ es la relación de transición. Sea $\phi(s)$ la fórmula que representa los estados que cumplen con la *propiedad de seguridad*. Con una cota k , el BMC se encarga de desenrollar el sistema k veces convirtiéndolo en una condición de verificación $B(k)$, donde:

$$B(k) = I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge \bigvee_{i=0}^k \neg\phi(s_i)$$

Es decir, partiendo de un estado inicial $s_0 \in S_0$ y habiendo realizado k transiciones correctas, tenemos una secuencia de estados válidos; si en alguno de estos se cumple la negación de la propiedad ϕ , entonces la fórmula es satisfacible. Luego un solucionador SMT puede proporcionar alguna asignación (valores para las variables del programa) que la satisfaga, y a partir de ella se construye un contraejemplo, el cual es una traza de estados $s_0 s_1 \dots s_k$ tal que $s_0 \in S_0$ y todas las transiciones $T(s_i, s_{i+1})$ son válidas para $0 \leq i \leq k$. Si la fórmula es insatisfacible, solo podemos asegurar que la propiedad se cumple en k o menos pasos.

Para asegurar completitud, ESBMC utiliza la k -inducción [7]. Observar el siguiente algoritmo:

$$\text{kind}(P, k) = \begin{cases} P \text{ contiene un bug,} & \text{si } B(k) \text{ es satisfacible} \\ P \text{ es correcto,} & \text{si } B(k) \vee [F(k) \wedge S(k)] \text{ es insatisfacible} \\ \text{kind}(P, k+1), & \text{en otro caso} \end{cases}$$

donde:

- $B(k)$ es la fórmula BMC estándar. Si es satisfacible, existe un contraejemplo de longitud $\leq k$.
- $F(k)$ es la **condición forward**, que verifica si se alcanzó la cota de completitud insertando *unwinding assertions* después de cada bucle. Si $F(k)$ es insatisfacible, todos los bucles se desenrollaron por completo y no hace falta continuar.
- $S(k)$ es el **paso inductivo**. Está formulado de manera tal que es insatisfacible si siempre que la propiedad ϕ se cumple para un desenrollado de tamaño k , se cumple también para uno de tamaño $k+1$.

El algoritmo sobre-aproxima el comportamiento de los bucles: antes de entrar a un bucle, se asignan valores no deterministas (havoc) a todas las variables que

el bucle puede modificar, permitiendo que tomen cualquier valor posible dentro de su dominio. Además, se fuerza la entrada al bucle mediante una *assumption* que garantiza que la condición del bucle es verdadera. Dado que esta técnica sobre-aproxima los estados alcanzables, puede producir falsos positivos, pero si el paso inductivo y el caso base son ambos insatisfacibles, se puede concluir que el programa es correcto para cualquier número de iteraciones.

6. Algunos casos de estudio

La herramienta ESBMC cuenta con usos importantes en la industria y posee reconocimientos en competencias de verificación de software como SV-COMP. Nos referiremos a algunos casos de uso de la herramienta.

Un caso de estudio notable fue en 2024 [12], sobre Realm Management Monitor (RMM) de Arm Confidential Computing Architecture [1]. Para probar la seguridad real de la herramienta RMM, los ingenieros tenían pensado utilizar CBMC³, pero decidieron también ponerla a prueba ante ESBMC. Esta decisión fue en parte por el éxito de la última en competencias de verificación de software, donde mostró mejor rendimiento que CBMC en prácticamente todas las áreas. Gracias a ello, ESBMC fue capaz de detectar 23 nuevos fallos en RMM no detectados por CBMC. No obstante, al comparar los tiempos de ejecución de estas herramientas se observó que en el modo de verificación multi-propiedad ESBMC era bastante más lenta, incluso llegando a sobrepasar el temporizador de 5000s principalmente debido a múltiples llamadas independientes a su solucionador SMT. CBMC aprovecha el método de SAT-solving incremental (que permite reutilizar instancias ya resueltas) con MiniSat para resolver el mismo tipo de verificación en menos tiempo.

Otro caso de estudio reciente fue la utilización de ESBMC-Python para verificar el protocolo de consenso de la blockchain de Ethereum⁴. Gracias a esto, se detectó una división por cero que podía ocurrir en una función llamada 'integer_squareroot'. Este error podía llevar a interrupciones de servicio y facilitar ataques a la red. La causa era un posible *unsigned integer overflow*⁵ que fue detectado por ESBMC, y ya fue corregida con una pequeña modificación al código.

Por último, destacamos la creación y evaluación de la herramienta ESBMC-Arduino que combina el verificador ESBMC con la plataforma de hardware de Arduino [10]. En este caso, se utiliza para mejorar la seguridad del código C de Arduino. Esta colaboración promete ser particularmente útil para sistemas embebidos críticos, ya que mejora el análisis de seguridad y promueve prácticas de desarrollo basadas en contratos. También se considera valiosa para la docencia y la investigación avanzada en verificación formal y seguridad de sistemas embebidos.

³ <https://www.cprover.org/cbmc/>

⁴ <https://github.com/ethereum/consensus-specs>

⁵ Cuando se incrementa en 1 un entero no signado del valor máximo, este toma el valor mínimo (0).

7. Comparación con CBMC

Como se mencionó anteriormente, ESBMC comparte un origen común con CBMC. Desde aquel entonces, ambas herramientas siguieron sus caminos propios, por lo que compararlas en su estado actual resulta interesante. Algunas de las diferencias más notables son mencionadas a continuación.

ESBMC dio un paso adelante al utilizar el front-end Clang para C, C++, CUDA y CHERI-C, mientras que CBMC todavía depende de sus propios analizadores sintácticos de C y C++, que están por detrás de Clang en características modernas de estos lenguajes. Más aún, ESBMC provee soporte integrado para Python 3.10+, Solidity, Kotlin/Java y próximamente Rust, mientras que CBMC solo verifica programas de C/C++ en la herramienta principal. En el back-end, ESBMC codifica los programas directamente en fórmulas SMT lo que permite aprovechar características de solucionadores avanzados y CBMC se basa principalmente en fórmulas SAT junto a su solucionador integrado MiniSat.

En cuanto a técnicas de demostración, ESBMC implementa BMC incremental, k-inducción y k-inducción bidireccional en una sola invocación, combinando comprobaciones de caso base, condición de avance y paso inductivo en una sola ejecución. La k-inducción de CBMC requiere tres pasadas separadas (construcción de CFG, anotación del programa, verificación) y carece de la condición de avance necesaria para demostrar que se han cubierto todos los estados alcanzables, basándose en su lugar en el desenrollado acotado.

En programas concurrentes, ESBMC explora las intercalaciones de hilos bajo verificación acotada, con estrategias de exploración *lazy* y *schedule-recording*, y una pasada de reducción de orden parcial (POR) que elimina las intercalaciones independientes. Esto suele ofrecer un mejor rendimiento que la codificación de fórmula única de CBMC para todo el programa concurrente cuando hay muchos puntos de intercalación.

8. Verificación del protocolo de consenso de la Ethereum blockchain

Entre los casos de estudio antes mencionados, analizaremos en detalle la verificación que se llevó a cabo sobre la Ethereum Consensus Specification [6]. Esta especificación contiene las implementaciones de una gran cantidad de funciones en Python, las cuales fueron verificadas individualmente con ESBMC. Se encontró una vulnerabilidad aritmética en una función llamada `integer_squareroot` la cual era una posible división por cero que sería crítica en un sistema de blockchain tan importante.

Código Original

```
# archivo int_sqrt.py
def integer_squareroot(n: uint64) ->
    uint64:
    """
        Return the largest integer 'x'
        such that 'x**2 <= n'.
    """
5: x = n
   y = (x + 1) // 2
   while y < x:
       x = y
       y = (x + n // x) // 2
   return x
```

Código Corregido

```
# archivo int_sqrt.py
def integer_squareroot(n: uint64) ->
    uint64:
    """
        Return the largest integer 'x'
        such that 'x**2 <= n'.
    """
   if n == UINT64_MAX:
       return UINT64_MAX_SQRT
   x = n
   y = (x + 1) // 2
   while y < x:
       x = y
       y = (x + n // x) // 2
   return x
```

Al ejecutar el comando `esbmc int_sqrt.py` con las opciones `--function integer_squareroot` y `--k-induction` sobre el código original se obtiene el siguiente output (simplificado para el lector):

```
VERIFICATION FAILED [Counterexample]
State 1 line 5 column 4: x = UINT64_MAX
State 2 line 6 column 4: y = 0
State 3 line 8 column 8: x = 0
State 4 line 9 column 8: Violated property
    file int_sqrt.py line 9 column 8 function
        integer_squareroot
    division by zero
x != 0
```

Es decir que ESBMC logró identificar la falla generando el contraejemplo matemático exacto que la causaba ($n = \text{UINT64_MAX}$, que resultaba en un *overflow*), facilitando la implementación del código corregido.

9. Conclusiones particulares

ESBMC demuestra ser una herramienta sólida y fácil de utilizar para la verificación de software. Su arquitectura le permite expandirse a nuevos lenguajes de programación de manera sencilla, y su compatibilidad con múltiples solucionadores SMT le da una flexibilidad que otras herramientas no proveen. Las innovaciones que presenta en cada nueva versión y su buen desempeño en competencias de verificación despiertan interés en la industria, donde los casos de uso son cada vez más frecuentes.

Los desarrolladores tienen como objetivo seguir mejorando la eficiencia y calidad de los algoritmos existentes, ya que ESBMC tiene algunas debilidades como la lentitud en la verificación multi-propiedad y los casos en donde el resultado es erróneo (falsos negativos y positivos). También se enfocan en aumentar la cantidad (y completitud) de lenguajes soportados.

Referencias

1. Arm: Realm management monitor. <https://developer.arm.com/documentation>
2. Beyer, D.: Competition on Software Verification and Witness Validation: SV-COMP 2023. In: Proceedings of the 29th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2023). Lecture Notes in Computer Science, vol. 13994, pp. 495–522. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30820-8_29
3. Cordeiro, L., Fischer, B., Marques-Silva, J.: SMT-Based Bounded Model Checking for Embedded ANSI-C Software. In: 2009 IEEE/ACM International Conference on Automated Software Engineering. pp. 137–148. IEEE (2009). <https://doi.org/10.1109/ASE.2009.63>
4. Cordeiro, L., Fischer, B., Marques-Silva, J.: SMT-based bounded model checking for embedded ANSI-C software. IEEE Transactions on Software Engineering **38**(4), 957–974 (2012). <https://doi.org/10.1109/TSE.2011.59>
5. esbmc: Sitio web oficial de esbmc. <https://esbmc.org>, Último acceso: 2026-05-29
6. Farias, B., Menezes, R., de Lima Filho, E.B., Sun, Y., Cordeiro, L.C.: ESBMC-Python: A Bounded Model Checker for Python Programs. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024). pp. 1836–1840. ACM (2024). <https://doi.org/10.1145/3650212.3685304>
7. Gadelha, M.R., Monteiro, F., Cordeiro, L., Nicole, D.: Esbmc v6.0: Verifying c programs using k-induction and invariant inference. In: Beyer, D., Huisman, M., Kordon, F., Steffen, B. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 209–213. Springer International Publishing, Cham (2019)
8. Gadelha, M.R., Monteiro, F.R., Morse, J., Cordeiro, L.C., Fischer, B., Nicole, D.A.: ESBMC 5.0: An Industrial-Strength C Model Checker. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE 2018). pp. 888–891. ACM (2018). <https://doi.org/10.1145/3238147.3240481>
9. Li, X., Song, K., Gadelha, M.R., Brauße, F., Menezes, R.S., Korovin, K., Cordeiro, L.C.: ESBMC v7.6: Enhanced model checking of C++ programs with clang AST. Science of Computer Programming **246**, 103336 (2025). <https://doi.org/10.1016/j.scico.2025.103336>
10. Silvestrim, R.G., Trigo, F.V., Rocha, W., Vieira, M.R.S., Junior, J.V., Mendes, O.d.C., Menezes, R., Cordeiro, L.: Towards Integrity and Reliability in Embedded Systems: The Synergy of ESBMC and Arduino Integration. In: 2023 XIII Brazilian Symposium on Computing Systems Engineering (SBESC). pp. 1–6. IEEE (2023). <https://doi.org/10.1109/SBESC60926.2023.10324098>
11. Song, K., Gadelha, M.R., Brauße, F., Menezes, R.S., Cordeiro, L.C.: ESBMC v7.3: Model Checking C++ Programs Using Clang AST. In: Formal Methods: Foundations and Applications (SBMF 2023). Lecture Notes in Computer Science, vol. 14414, pp. 141–152. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-49342-3_9
12. Wu, T., Xiong, S., Manino, E., Stockwell, G., Cordeiro, L.C.: Verifying Components of Arm Confidential Computing Architecture with ESBMC. In: Static Analysis (SAS 2024). Lecture Notes in Computer Science, vol. 14995, pp. 451–462. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-74776-2_18